VCF-RDFizer: From a VCF File to Explicit, Verifiable, and Policy-aware Semantic Genomic Data

Elias Crum[ORCID1,2*], Bart Buelens[ORCID2], Ruben Taelman[ORCID1], Gökhan Ertaylan[ORCID2]

[1]IDLab, Department of Electronics and Information Systems, Ghent University – imec, Technologiepark-Zwijnaarde 116, 9052, Belgium.
[2]Digital Biological Systems, Flemish Institute for Technological Research (VITO), Boeretang 200, 2400, Belgium.

*Corresponding author(s). E-mail(s): elias.crum@ugent.be;

Abstract

Background: The Variant Call Format (VCF) is compact, widely used, and readily parsed, but workflows that combine genomic observations with changing evidence, clinical context, or data-use policies rely on relationships maintained outside the file. Making such relationships explicit could support variant reanalysis, pharmacogenomic decision support, and consent-aware reuse, provided that the converted representation remains verifiable against the source. We present VCF-RDFizer, a reusable command-line tool that converts VCF into the Resource Description Framework (RDF) using the public VCF Core vocabulary.
Results: We evaluate source fidelity, linked and policy-aware reuse, scalability, retrieval performance, and semantic coverage against four other executable VCF-to-RDF converters. Across 78 validation runs, all 1,006 applicable SPARQL comparisons agree with answers derived from the source VCF. Source-comparison queries detect 96 of 113 injected faults, while deeper SHACL constraints detect the remaining 17. A linked workflow connects variant observations to ClinVar classifications in 81 ACMG secondary-finding genes and to population frequencies. Simulated consent policies define which VCF data subsets were available to different requester roles through RDF release views. Linked-data queries over these views return the same participant–variant–gene matches as a conventional bcftools workflow across gene-span slices from five heterogeneous VCFs, a 104-participant cohort, and two complete single-sample VCFs. We compare five VCF-to-RDF converters on identical inputs and show that only VCF-RDFizer reproduces source-derived answers to all eight record, genotype, and header

summary questions, although VCF-RDFizer's broader representation increases conversion time and output size. Direct VCF processing is faster for one-pass analyses and indexed regions up to 100 kb, whereas indexed RDF was faster for 10 Mb regions. For repeated whole-VCF questions, indexed RDF recovers its setup cost after about 11 separately executed questions.
Conclusions: VCF-RDFizer provides a source-verifiable RDF representation that supports external knowledge integration and policy-aware genomic data reuse. Our evaluation demonstrates these capabilities while quantifying conversion, storage, and indexing costs. The results establish where RDF complements direct VCF processing and provide a foundation for more advanced linked genomic workflows.

Keywords: Genomic variation, Variant Call Format, RDF, SPARQL, Semantic validation, Data-use policies

1 Background

The Variant Call Format (VCF) is widely used to store and exchange genomic variant data in research and clinical applications, including pharmacogenomics and precision medicine [1–3]. VCF is compact, structured, and supports identifiers, reference information, and a wide range of annotations [1, 4]. However, many biomedical questions combine genomic variant observations with clinical records, external knowledge, sample metadata, and conditions governing data use [5–7]. The central challenge is therefore not reading VCF, but representing relationships that extend beyond the file in a form that can be reused across systems and questions. We consider three generalized use cases that illustrate this need.

(1) Clinical variant interpretation and reanalysis. Clinical interpretation combines genomic observations with clinical assertions, population frequencies, gene–disease associations, and patient phenotypes, all of which can change over time [8, 9]. Reanalysis can consequently identify new pathogenic or likely pathogenic findings and downgrade earlier classifications [10]. The genomic observation and the supporting evidence therefore follow different lifecycles, while the relationship between observation and evidence is often maintained only in annotations or application-specific databases.

(2) Pharmacogenomic clinical decision support. Pharmacogenomic decision support connects genomic variant observations to an interpreted phenotype, medication, indication, and clinical guidance. For example, CYP2C19-guided clopidogrel recommendations depend on both metabolizer phenotype and clinical indication [11]. Current systems still face challenges in representing genomic results as discrete data, integrating them with electronic health records, and maintaining the knowledge used for interpretation [12, 13].

(3) Consent-aware genomic data reuse. Secondary use adds a governance dimension to the same integration problem. Participant consent and associated data-use conditions define which research purposes are permitted, and those permissions can vary across participants, datasets, and intended uses [7, 14]. The GA4GH Data Use Ontology (DUO) and Machine Readable Consent Guidance provide machine-readable

vocabularies and structures for representing such conditions, while decisions about whether a specific use is authorized remain subject to institutional, ethical, and legal oversight [7, 14, 15]. Because genomic data are sensitive and potentially identifying, release decisions for participants, genomic regions, or individual variants remain tied to the consent and governance conditions under which the data may be used [16, 17].

None of these tasks is impossible with the current VCF representation. Variant-analysis frameworks, clinical systems, and data-access infrastructures already address important parts of the workflows [7, 12, 18]. The remaining challenges increasingly concern interoperability, reproducibility as knowledge changes, and the computational and operational cost of connecting independently maintained systems. VCF was designed to represent genomic variation, genotypes, and associated annotations rather than to serve as a shared model linking source observations to longitudinal evidence, clinical context, and data-use conditions [19]. Downstream systems therefore often add such relationships outside the source representation. For example, GEMINI imports VCF variants into a relational database and augments them with resources such as ClinVar, dbSNP, and population annotations to support integrated queries [18]. Such application-specific integration motivates a shared representation in which relationships between source observations and independently maintained information can be expressed explicitly and reused across workflows.

Knowledge graphs provide a way to connect entities and relationships across independently maintained data sources. RDF provides a standardized graph model in which resources are identified by IRIs, while Linked Data practices and SPARQL support cross-resource linking and querying [20–23]. The use of persistent identifiers and shared semantics also supports the interoperability and reuse goals of the FAIR principles [24]. For genomic data, RDF's explicit identifiers and relationships allow source observations to remain distinct from biological variants, external evidence, clinical context, and associated data-use policies. SPDI (Sequence:Position:Deletion:Insertion) representations can support joins across files and resources such as dbSNP and ClinVar [25–27], while ODRL (the W3C Open Digital Rights Language) and DUO can represent policy conditions over identified resources [14, 28]. RDF therefore provides a reusable representation for integration, but reliable use still depends on defined identity rules, interpretation processes, and policy enforcement [28, 29].

Several semantic-genomics initiatives and VCF-to-RDF converters already explore this foundation. Many were developed for focused applications or as components of larger pipelines and consequently represent only the VCF information required for those tasks [30–34]. Existing tools also differ in their RDF models, treatment of headers and genotypes, and extension mechanisms [31, 33, 35–37]. Supplementary Table S1 summarizes documented capabilities and the evidence behind each assessment. Non-RDF frameworks such as Hail, GEMINI, and TileDB-VCF, and standards such as Beacon, htsget, and FHIR Genomics, address complementary parts of variant analysis, retrieval, and clinical exchange [18, 38–42]. Our aim is not to replace such systems but to produce sufficiently complete RDF for multiple downstream links and queries without redesigning the representation for each application.

Producing RDF is only a first step. Evaluating the resulting representation also involves verifying its fidelity to the source VCF and measuring the costs of conversion

and downstream use. SPHN Schema Forge, FAIR-Checker, and a FHIR RDF enrichment pipeline provide precedents for assessing semantic transformations alongside the validation and information requirements of their intended workflows [43–45].

We present VCF-RDFizer, a tool that converts VCF into RDF using the public VCF Core vocabulary [46]. The vocabulary was developed separately [47], while the present study focuses on the converter, its representation choices, and the potential for downstream use of the generated RDF.

The evaluation of our tool investigates whether the RDF representation can preserve verifiable source information, support links to independently maintained knowledge, and enable policy-aware data reuse without modifying the converted VCF data. A linked research workflow combines genomic observations with ClinVar classifications, population frequencies, and simulated data-use policies [27]. Requester-specific query results are compared with an equivalent conventional workflow that operates directly on the VCF files, while separate experiments assess conversion fidelity, validation sensitivity, scalability, and retrieval costs.

Three research questions guide the evaluation:

RQ1: Which aspects of VCF information are preserved in RDF and can be validated against the source?

RQ2: How does the RDF representation support linking genomic observations to external knowledge and enable policy-aware querying?

RQ3: What are the computational costs of RDF representation choices, and when can the additional capabilities of VCF as RDF justify those costs?

Together, the experiments evaluate technical requirements shared by clinical variant reanalysis, pharmacogenomic decision support, and consent-aware genomic data reuse; they are intended to inspire future applications, not to offer directly implementable solutions.

2 Implementation

VCF-RDFizer is a command-line tool for converting VCF into RDF and validating selected properties of the result against the source VCF file.[1] A pinned Docker image contains the conversion toolchain, and tagged versions are also distributed through PyPI and conda-forge [48–50].

2.1 Representation of VCF

VCF-RDFizer represents the source VCF document rather than only an interpreted biological variant. The VCF Core vocabulary guides the representation of file and header assertions, variant records, alleles, and sample/genotype assertions (Figure 1) [46, 47]. The vocabulary aligns VCF-specific structures with FALDO, the Sequence Ontology, HERO, and selected GA4GH VRS concepts [51–55].

VCF-RDFizer assigns each VCF record a deterministic IRI derived from its source file identifier and record position. For example, two VCF files reporting the same genomic alteration receive distinct record IRIs because each identifier incorporates the originating file (see the VCF Core Vocabulary companion for an example). Each

[1] Source code and documentation: https://github.com/ecrum19/VCF-RDFizer.

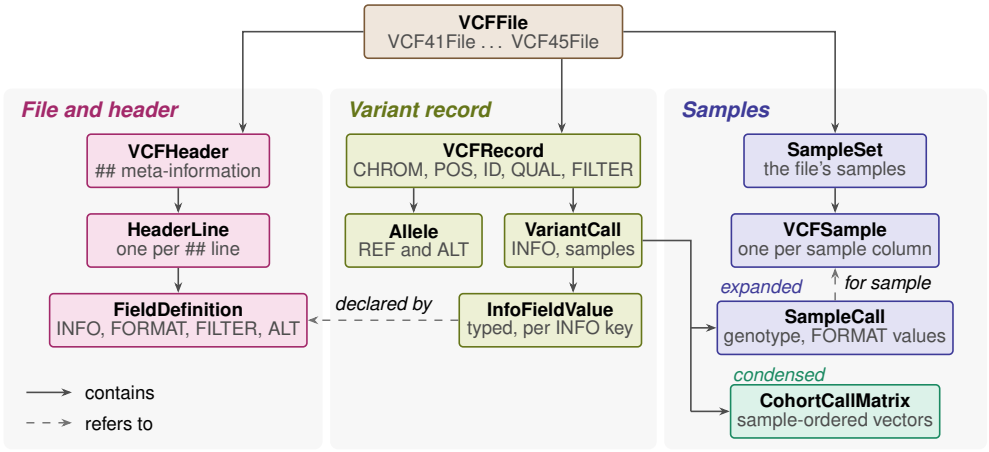


Fig. 1 Central VCF Core terms emitted by VCF-RDFizer. Each VCF file links its header declarations, variant records, and samples. Records distinguish alleles from calls, while INFO values retain links to their header definitions. The *expanded* profile exposes individual sample calls, whereas the *condensed* profile stores sample-ordered vectors to reduce graph size. Both preserve VCF source context. Supplementary Figure S1 presents an expanded version of the model.

observation therefore retains its own quality scores, annotations, and provenance, while a separate linkset, a file of links kept apart from the converted VCF graph, can connect both records to a shared variant identifier.

VCF-RDFizer provides two sample representation profiles that balance direct query access against graph size. The *expanded* profile represents sample calls and FORMAT values as directly accessible RDF resources, whereas the *condensed* profile stores values in sample-ordered vectors that require decoding to retrieve individual genotypes. Both profiles preserve file, header, and record context but differ in storage requirements and query performance. Supplementary Section S2.1 explains these trade-offs, while Supplementary Section S2 provides the complete model, capability matrix, and scaling analysis.

2.2 Conversion and validation

Conversion follows the workflow in Figure 2 and is described in detail in VCF-RDFizer's documentation. The process first derives tabular logical sources (TSV formatted) from the VCF input file. RML mappings, declarative rules that map tabular data to RDF, are executed by RMLStreamer and produce the record backbone of nine core triples per record, while direct emitters, code in the converter that writes triples without mappings, add typed values, header attributes, and sample structures that require per-value handling [56, 57]. The implementation is therefore hybrid: editing the RML mappings changes only the record backbone, not the sample, header, and typed-value triples. Output is appended incrementally to plain or gzip-framed N-Triples. HDT (a compressed binary RDF format that can be queried without decompression), COTTAS (compressed triple tables stored as Parquet files), and additional

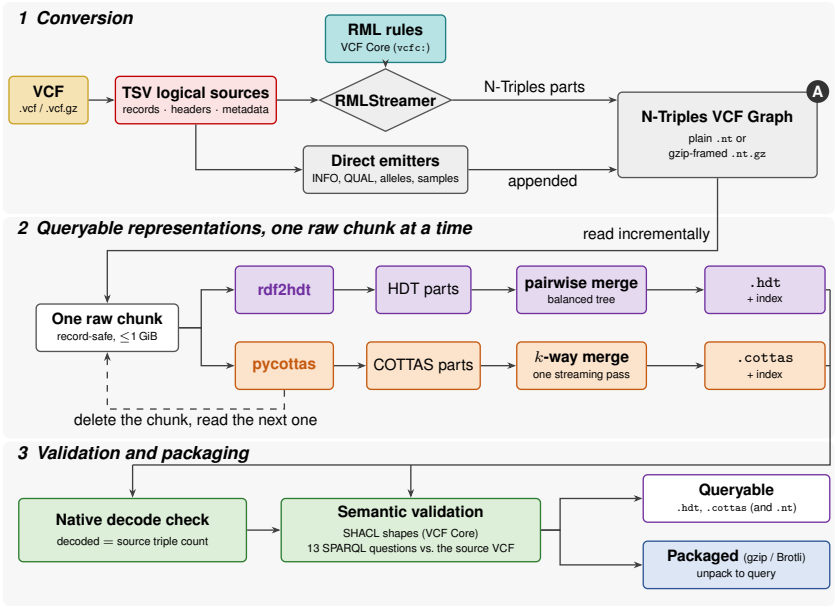


Fig. 2 The VCF-RDFizer pipeline. (1) Conversion: the VCF is split into tab-separated tables (logical sources); declarative RML mapping rules, executed by RMLStreamer, and the converter's own emitters turn them into RDF triples, appended incrementally as plain or gzip-framed N-Triples (one triple per line; gzip-framed output is a sequence of gzip members). (2) Optional queryable representations: HDT and COTTAS, compressed queryable formats, are built one chunk of complete N-Triples lines at a time before merging and indexing. (3) Validation and packaging: decoding checks on each artifact's triple count, the source-comparison queries `Q1`–`Q13`, and enabled SHACL profiles assess different properties of the result.

gzip or Brotli compressions are other optional artifacts that can be produced [58–60]. Native decoding checks compare each HDT or COTTAS triple count with that of the N-Triples source graph. Construction and query-path details are provided in Supplementary Section S2.2.

Validation is built into VCF-RDFizer rather than run as a separate analysis: `-validate` checks each input after conversion, `-mode validation` checks an existing graph against its source VCF, a failed check marks the run as failed, and the experiments in Section 3 use this same validator. Validation compares 13 SPARQL *source-comparison queries* (`Q1`–`Q13`) with answers independently derived from the source VCF using cyvcf2 and bcftools for exact FILTER text [4, 19, 61]. The source-derived answers serve as the *oracle*. The 13 queries form three layers, each targeting potential errors that the previous layer cannot detect. The queries and oracle were developed using a purpose-built test VCF with records and fault-injections designed to cover the core features of VCF. `Q1`–`Q4` summarize record positions, allele classes, transitions and transversions, and FILTER values. `Q5`–`Q8` assess genotypes, allele counts, file metadata, and header declarations. `Q9`–`Q10` count triples per predicate and resources per class, while `Q11`–`Q13` compare hashed summaries of record, INFO, and FORMAT values (Supplementary Table S7). The queries produce counts, histograms,

and hash-bucket summaries that the oracle can independently compute from the VCF. Normalized results are compared for exact agreement, although the summaries do not establish exhaustive value-by-value equivalence. Figure 3 shows a slightly simplified version of `Q3` as well as its cyvcf2 oracle. All full queries and corresponding oracles used for all experiments can be found on the results site. Fourteen preflight integrity checks and SHACL constraints assess additional graph properties [62].[2] Along with the 13 queries, a default SHACL profile checks cardinality and datatypes, while two deeper SHACL profiles assess value agreement, such as genotype text against called alleles, and index uniqueness. Table 1 summarizes the validation coverage, and Supplementary Section S5 details the queries, normalization, constraints, and limitations.

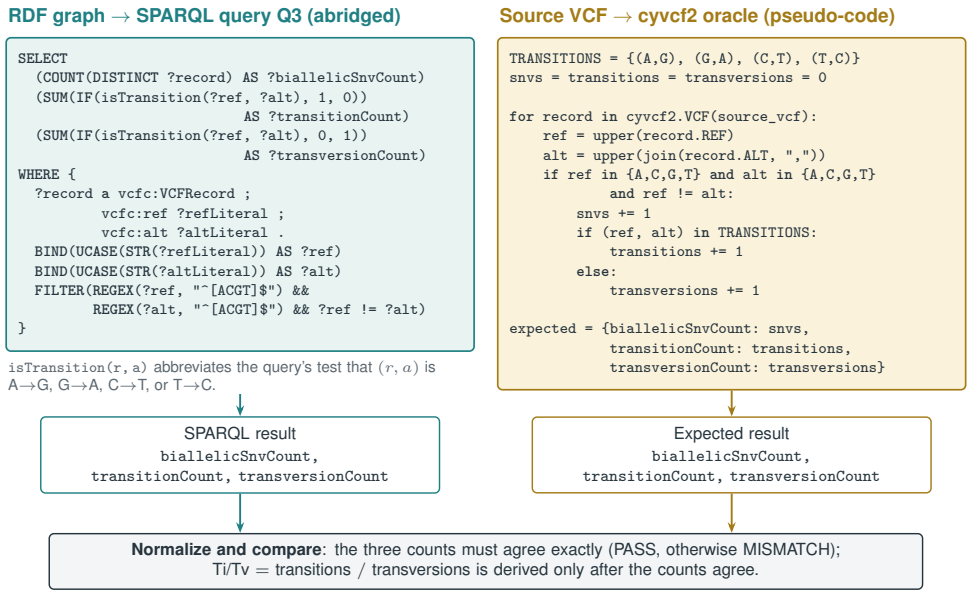


Fig. 3 Example of validating RDF query results against the source VCF. SPARQL query `Q3` (left, abridged) and its cyvcf2 oracle (right, pseudocode) independently count biallelic SNVs, transitions, and transversions from the RDF graph and source VCF, respectively. The three counts are normalized and compared for exact agreement before calculating the Ti/Tv ratio. The example illustrates how source-comparison queries assess conversion fidelity.

2.3 Linking and policy extensions

Two plug-in functionalities within VCF-RDFizer extend the generated VCF graph without modifying it.

`vcf-rdfizer-link` executes linkers, declared rules that connect records or alleles to external identifiers, and writes the resulting relationships to separate linksets with coverage reports. The evaluated linkers connect eligible alleles to SPDI identifiers and

[2]Every preflight check and SHACL shape, with its exact text: https://ecrum19.github.io/vcf-rdfizer-testing/#rq1-checks.

Table 1 VCF representation and validation coverage. For each VCF component, the table identifies the RDF representation, validation checks, and their limitations. `Q1`–`Q13` are source-comparison queries (Section 2.2), with `Q9`–`Q10` counting predicates and classes across the graph. SHACL shapes assess graph constraints, while test VCFs exercise specific cases. The linked workflow is evaluated in Section 3.2.

Information	Representation	Checked by	Limit
File and header	Versioned file, declarations, and linked metadata	`Q7`, `Q8`, shapes	Attribute–value agreement needs the deeper profiles
Record and alleles	Core fields, ordered alleles, typed and explicitly missing values	`Q1`–`Q4`, `Q11`, shapes	Summary digest, not exhaustive equality
INFO	Raw string plus optional typed, structured values	`Q11` (raw), `Q12`, shapes	Preserved, not universally interpreted
Samples and FORMAT	Expanded sample calls or condensed vectors	`Q5`, `Q6`, `Q13`	Condensed values require decoding
VCF versions	VCF 4.1–4.5 file classes and retained newer fields	Fixtures	Some constructs retained lexically only
Links and policies	Separate linksets and policy-defined release views	Linked workflow	Identity assumptions and simulated policies

variant calls to overlapping genes. The SPDI linker harmonizes contig names, such as `chr1` and `1`, and removes shared REF and ALT bases so matching simple variants receive the same identifier. However, equivalent insertions or deletions within repeat regions may still receive different identifiers because the linker does not shift alleles within repeats [25].

The third use case addresses releasing genomic data with different requesters for different use purposes. The `vcf-rdfizer-policy` plug-in defines and evaluates ODRL permissions and prohibitions over files, genomic regions, named variants, and gene-linked records, using standardized use purposes from DUO [14, 28]. Access is denied unless explicitly permitted, and prohibitions override conflicting permissions. Withholding a file or record also removes dependent resources, such as headers, alleles, and sample calls. For each requester, the plug-in generates a *release view* containing only permitted RDF data. Each view is validated against the policy and a record list derived from the source VCF. The evaluation uses simulated consent policies and does not provide anonymization or enforce additional obligations, such as deleting released data [63]. Operational deployment would also require authentication, access controls, auditing, and other security safeguards that are not implemented in this demonstration [64]. Supplementary Section S6 further details linking, policy evaluation, and release validation.

3 Evaluation and Results

The experiments assess representation fidelity and validation (RQ1), a linked and policy-aware workflow (RQ2), and scalability, retrieval, and comparative semantic coverage (RQ3). Each subsection below first describes how its question was tested and then reports the test results. The *core benchmark* comprises 143 runs in 12 experiment families covering conversion, scaling, storage, input coverage, configuration, validation, and query cost, all executed with the published VCF-RDFizer v3.1.0 release on two hosts with identical hardware. The linked workflow (Section 3.2), consumer WGS validation (Section 3.1), shared-input converter comparison, index storage, and regional and large-graph retrieval (Section 3.3) are separate experiments using VCF-RDFizer v3.3.1. Additional v3.1.0 runs cover default-profile fault injection and N-Triples-only query costs; none is counted in the core benchmark's totals. Version 3.3.1 includes the evaluated linkers and policy plug-in; on the same 100,000 HG005 records, v3.1.0 and v3.3.1 produced identical sorted triples. Supplementary Section S3.1 lists the core benchmark's experiment families, and Section S3 provides the release ledger, hosts, dependencies, and archived evidence for every experiment. An interactive results browser presents detailed, interactive views of this study's data organized by research question.[3]

The evaluation uses public VCF files from the Personal Genome Project, Genome in a Bottle (GIAB), precisionFDA Truth Challenge V2, 1000 Genomes Phase 3, and HGSVC2 [65–69]. The experiment repository documents the exact inputs, provenance, and download sources. The GIAB HG005 benchmark VCF is used for validation, scaling, retrieval, and converter comparisons. Subsets of the 1000 Genomes chromosome-20 call set support sample-count experiments and converter comparisons. Two consumer whole-genome VCFs, `NG131FQA1I` (Dante Labs) and `NB72462M` (Nebula Genomics), support consumer WGS validation (Section 3.1) and Scenario 4 (Section 3.2.1), respectively. Supplementary Table S5 lists all inputs.

3.1 RQ1: Source comparisons and structural validation provide complementary evidence

3.1.1 Evaluation design

We organize the evaluation into five groups of benchmark assessments (labeled A–E). For all benchmarks, we assess the same 13 SPARQL queries (`Q1`–`Q13`) and SHACL checks compared with input-specific oracle results computed directly from the input VCF. **A: Core benchmark validation** assesses 11 difficult-input fixtures (such as zero records, sites only, and a truncated gzip), a 1,000-line fixture across ten different configurations and seven operating modes of VCF-RDFizer, a 9,986-record single-sample VCF input with both sample profiles (expanded and condensed) converted into three different RDF artifact formats (N-Triples, HDT, COTTAS), and slices of increasing numbers of HG005 VCF records, up to 100,000 records. **B: Artifact decoding** checks whether HDT and COTTAS artifacts decode to the triple count of their N-Triples source graph. **C: Consumer WGS validation** assesses the

[3] https://ecrum19.github.io/vcf-rdfizer-testing/

first 250,000 records of `NG131FQA1I`, a publicly shared personal-genome VCF generated by the commercial consumer sequencing provider Dante Labs. **D: Large-graph query agreement** evaluates one million HG005 records and the complete HG005 VCF using different query engines: N-Triples (QLever), HDT (Comunica-HDT), and COTTAS (Comunica-COTTAS) for execution, omitting preflight checks and SHACL. **E: Fault-injection sensitivity** measures detection of 113 deliberate changes to otherwise correct RDF graphs, using queries alone, queries with default SHACL shapes, and queries with all three SHACL shape profiles. For reproduction, A, B, and E draw on the core benchmark scripts, with an additional default-profile run for E; C and D are separate experiments. Supplementary Section S5 details each group's scope, oracle dependencies, and execution limits. All results can be reproduced using the workflow described in the vcf-rdfizer-testing repository.

3.1.2 Results

The results of the validation experiments are summarized in Figure 4 with other interactive figures and details available on the companion website.

A: Core benchmark validation. All 78 validation runs completed, on fixtures and slices up to 17.1 million triples. All 1,006 applicable query comparisons agreed exactly with their source-derived counterparts after normalization; eight `Q5`/`Q6` comparisons were not applicable because the zero-record and sites-only fixtures contain no genotypes. All 1,160 internal consistency checks also passed: for example, the transition and transversion counts must sum to the biallelic SNV count. In 25 runs, QLever and the Comunica engines for N-Triples, HDT, and DuckDB-backed COTTAS returned identical normalized answers [70–74]. The remaining 53 runs used one engine: Comunica over N-Triples in 44 and QLever in nine. Default SHACL shapes reported no violations in 60 runs on graphs up to 0.96 million triples. Ten of the 11 difficult inputs converted and validated; the truncated gzip was correctly rejected before validation. Fixtures declaring VCF 4.1–4.5 converted successfully, although this does not establish coverage of every version-specific construct (Supplementary Section S4).

B: Artifact decoding. All 152 artifacts (118 HDT and 34 COTTAS) decoded to the same triple count as their N-Triples source graph. This checks artifact readability and count preservation; the query comparisons in A and D separately assess source agreement.

C: Consumer WGS validation. All 13 query results agreed on 58.2 million triples from the 250,000-record slice. Default SHACL shapes reported no violations across 117 batches of whole records, with 46 non-blocking recommendations for missing header attributes. This assessment extends the evaluation to a larger consumer VCF slice, not to its complete genome.

D: Large-graph query agreement. On both the whole genome and the one-million-record graph, all 13 QLever queries completed and their answers matched the input VCF-derived results. For the whole genome, QLever completed the evaluation in 10.8 minutes, excluding decoding and index construction time. On the one-million-record graph, Comunica-HDT queries completed `Q1`–`Q4` in 148.2 minutes before `Q5` reached the 60-minute timeout and the execution was halted. The Comunica-COTTAS queries completed `Q1`–`Q5` in 12.5 minutes before its engine crashed during `Q6` after

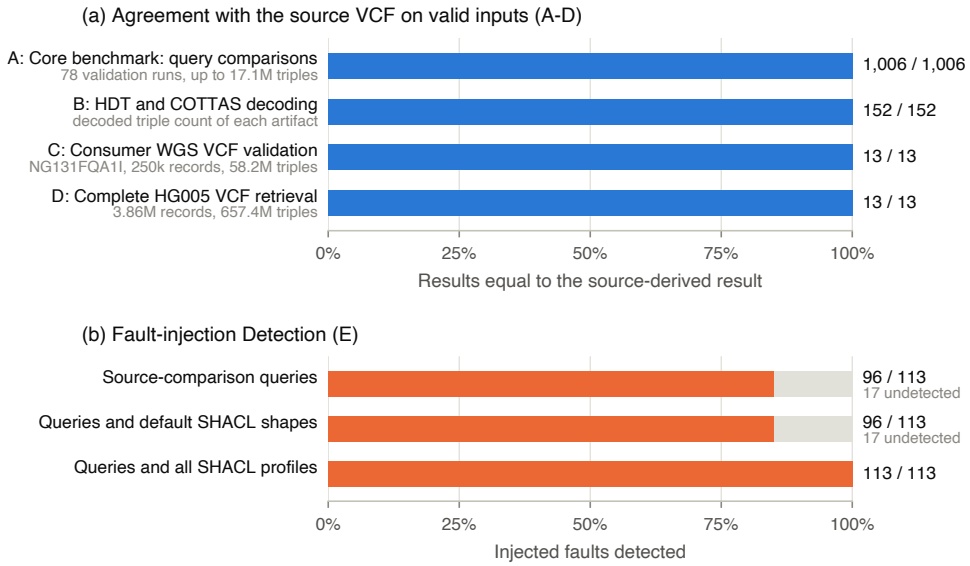


Fig. 4 Source agreement and detection of injected faults. Panel (a) summarizes core benchmark validation (A), artifact decoding (B), consumer WGS validation (C), and complete-HG005 query agreement (D). Decoding compares triple counts with the N-Triples source graph; the other rows compare query answers with the source VCF. Panel (b) shows detection of 113 injected faults (E) by queries alone, queries with default SHACL shapes, and queries with all three profiles. The deeper profiles detected every injected fault that the queries missed. Uppercase letter labels (A-E) identify the RQ1 benchmark groups.

another 7.3 minutes. The remaining queries did not execute and were reported as execution failures rather than observed answer disagreements. Of the queries that completed, all answers agreed with the VCF-derived results. The HDT and COTTAS Comunica engines did not complete all queries at the one-million-record scale, thus they were not tested on the whole genome scale. Execution limits and costs are reported in Supplementary Section S8.3 and figures representing this data are available on the companion webpage.

E: Fault-injection sensitivity. Source-comparison queries detected 96 of 113 injected faults. The other 17 changed relationships or values outside the query summaries, including called-allele links, sample indexes, phasing, and header attributes. The default shapes detected none of these 17 faults, but adding both deeper SHACL profiles detected all 17 and accepted both uncorrupted control graphs. The deeper profiles took 33.7 and 73.2 seconds on a graph of about 2,000 triples, compared with 1.7 seconds for the default profile. They load the whole graph into memory and were evaluated only on the small fault-injection graphs, thus this deeper validation is not yet viable for large graphs.

During testing, implementation defects identified and corrected in VCF-RDFizer are documented in Supplementary Section S10.

3.2 RQ2: Shared identifiers enabled reusable joins and policy-checked release views

3.2.1 Evaluation design

The three motivating use cases share a need to trace genomic observations to their source while linking them to independently maintained knowledge. We evaluate this capability through a single proof-of-concept workflow rather than implementing each application in full. The workflow uses the published 81-gene ACMG Secondary Findings (SF) v3.2 list to define a fixed, clinically motivated scope [75]. Within this gene set, the workflow links observed alleles to ClinVar classifications and evaluates which matches each requester role may receive under simulated consent policies. A match represents a participant–variant–gene tuple associated with a ClinVar record having at least one review star. All classifications are included, so most matches are benign, while pathogenic and likely pathogenic matches are reported separately. The workflow does not assess the additional criteria required for reportable clinical secondary findings because it is a demonstration of the data linking and policy annotation capabilities of VCF represented as RDF.

The RDF workflow, shown visually in Supplementary Figure S2, links converted VCFs to ClinVar and gene resources, materializes requester-specific views, and queries each view with SPARQL. The conventional workflow harmonizes contigs, annotates with bcftools, and applies the same case definition and simulated release rules in Python. The two workflows must return identical sorted participant, gene, contig, position, and allele tuples for every requester. Both workflows share preprocessing and the case definition, while downstream linking, policy evaluation, and query implementations remain separate.

Four evaluation scenarios assess heterogeneous VCF inputs, cohort integration, complete-genome scale, and fine-grained data release. *Scenario 1* uses records within the 81 ACMG gene regions from five single-sample VCFs (HG002, HG004, HG005, `NG131FQA1I`, and `NB72462M`). *Scenario 2* includes 104 unrelated, high-coverage 1000 Genomes participants [76]. *Scenario 3* uses the complete HG005 VCF. *Scenario 4* uses the complete `NB72462M` VCF to evaluate four distinct, non-empty requester-specific release views.

All scenarios use the ClinVar release from 2026-09-13 [77]. Scenario 2 also stores population frequencies once in a separate sites-only RDF graph, avoiding repeated panel-level INFO values across participants. A rare-variant query joins participant observations, ClinVar classifications, and population frequencies through SPDI, retaining matches with a panel allele frequency below 0.01.

Simulated consent policies define access for clinical care (CC), cardiovascular disease-specific research (DS), general research use (GRU), and, in Scenario 4, care by the participant's physician (OP). Each requester has one assigned purpose. The policies combine file-level permissions, withdrawals, gene restrictions, genomic regions, and individual variants. For example, in Scenario 3, the single participant's (HG005's) simulated consent permits only clinical care, so the DS and GRU requesters receive empty views and their queries return no matches. Supplementary Section S6 details the scenarios, policies, queries, and comparison procedures.

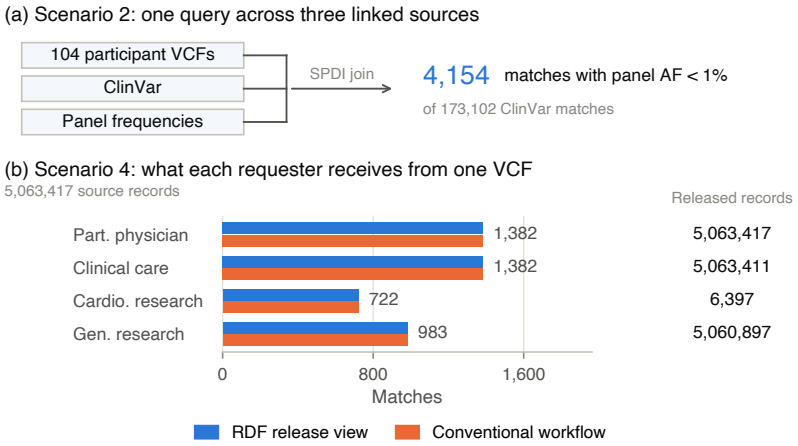
3.2.2 Results

Linked query matches. Across all four scenarios, the RDF and conventional workflows returned identical participant–variant–gene matches for every requester (selected results in Figure 5 and complete results in Supplementary Figure S3). Each match represents one participant, one observed variant with a ClinVar classification, and one of the 81 ACMG genes. Participants may contribute multiple matches, most of which were classified as benign. Scenario 2 additionally linked observations from 104 participants to ClinVar and a separate population-frequency graph containing 68,635 sites (6.5 million triples), using shared SPDI identifiers. Without a policy, the cohort produced 173,102 matches, of which 4,154 involved an allele with a panel frequency below 0.01 , which both agreed exactly with the conventional workflow. Assessing complete VCFs instead of subsets did not change the answers: the complete HG005 VCF (Scenario 3, 3.9 million records) returned the same 1,496 matches as its gene-region slice in Scenario 1, and the complete `NB72462M` VCF (Scenario 4) the same 1,382. Before any policy was applied, only eight matches, all in Scenario 2, were classified as pathogenic or likely pathogenic. These counts describe database matches, not unique participants or clinically adjudicated findings.

Requester-specific release and validation limits. Scenario 4 demonstrates selective release from the complete `NB72462M` VCF containing 5,063,417 normalized records. The participant's physician received every record, clinical care excluded six APOE records, general research excluded 2,520 restricted records, and cardiovascular research received only 6,397 panel records. Both workflows agreed on released-record counts and resulting matches in Scenarios 1, 3, and 4. All 13 materialized RDF views also passed the policy plug-in's release check, which confirms that each view was produced under the committed policy, contains no resource selected by an applicable prohibition and only resources covered by an applicable permission, and references no withheld resource; it then requires the released records to match exactly those obtained by evaluating the same policy on a graph built directly from the source VCF text.

However, Scenario 2 exposed a limitation in gene-based release rules. The RDF workflow retained 110 restricted structural-variant records for cardiovascular research and 58 for general research. The gene linker does not associate symbolic alleles, such as `<DUP>`, with genes, preventing the corresponding policy rules from selecting these records. Query matches remained unchanged because the analysis includes only explicit alleles. The release checks also missed the discrepancy because the policy evaluator and validator relied on the same incomplete gene links (more details in Supplementary Section S6.3). The result shows that agreement on query answers and passing policy checks do not establish complete release correctness when both depend on the same annotations.

Policy adaptation. Four rule changes tested whether existing policies could be updated without modifying the RDF policy engine. Restricting HFE C282Y to clinical care required a six-line policy edit using the variant's SPDI identifier. The rule also selected the HG002 record despite its different contig naming. The conventional workflow required corresponding code and configuration changes. The other three changes required similarly small data edits in both workflows (Supplementary Table S11). The



(c) Time spent in each workflow stage, Scenarios 2 and 4

Scenario	Convert (min)	Link (min)	Write view (min)	Query (s)
2	34.1	22.2	Not needed	174
4	88.7	16.5	54.5–69.7	187–233

Fig. 5 RDF representations of VCF support cross-resource queries and policy-aware data release. (a) SPDI identifiers link 104 participant VCFs to ClinVar and population frequencies, identifying 4,154 matches with allele frequency below 1%. (b) Simulated policies produce requester-specific views of one complete VCF. RDF results (blue bars) agree with conventional VCF results (orange bars) on match counts, exact participant–variant–gene tuples, and released-record counts. (c) Conversion and linking incur one-time costs, while release views require additional processing per requester. Query times are medians of three runs, with full costs in Supplementary Table S13. Together, the results demonstrate that RDF can support genomic data integration and selective reuse. However, matching query results do not guarantee correct policy release, as Scenario 2 retained 110 and 58 additional restricted records in the cardiovascular and general research views. Matches are not clinical findings.

results demonstrate reuse of shared variant identifiers for the evaluated policy change, rather than a general advantage in implementation effort.

Computational costs. Most linked-workflow processing occurred before query execution. In Scenario 3, conversion of HG005 and ClinVar required 55 minutes, linking required 8 minutes, and materializing, validating, and indexing the clinical release view required another 2.6 hours. Subsequent queries took 216–218 seconds. Across all scenarios, queries over non-empty views took 171–233 seconds, suggesting a substantial contribution from the fixed ClinVar join rather than scale-independent retrieval performance. In Scenario 4, materializing each selective view of the 1.2-billion-triple graph required 55–70 minutes. After shared preprocessing, the conventional workflow completed annotation, selection, and release in 4–91 seconds per scenario. In the RDF workflow, conversion and linking alone, before any release view was written, indexed, or queried, took 5.7–105 minutes per scenario: in every scenario more than 40 times as long as the complete conventional run for all requesters. For a single analysis, the conventional workflow was therefore far cheaper. The RDF workflow's cost lies in one-time preparation, which can be repaid if the linked graph and its release views are

reused for many questions, requesters, or policy revisions. Supplementary Figure S4 and Table S13 provide the complete measurements.

3.3 RQ3: Semantic coverage and practical cost depend on representation and workload

3.3.1 Evaluation design

VCF-to-RDF converters comparison. A shared-input comparison evaluates four other executable converters: JVarkit, TogoVar, SPARQLing Genomics, and BioInterchange. Each converter processes a 100,000-record HG005 input and a 10,000-record, 16-sample 1000 Genomes input with its most complete available options. Content questions `Q1`–`Q8` are translated to each vocabulary and compared with the same source-derived oracle, normalization, comparator, and QLever build used for VCF-RDFizer. A question is reported as not represented when the graph lacks the required information to answer the query. Queries for each converter can be found on the companion site. Conversion time, peak memory, and output size are measured over three replicates in pinned containers. Supplementary Section S7 provides the converter versions, execution commands, adapted SPARQL queries, and complete results.

VCF-RDFizer scaling experiments. Controlled scaling series vary one dimension at a time. The record-count series processes 10,000, 100,000, one million, and all HG005 records, while the sample-count series holds 10,000 cohort records fixed and varies the sample count from one to 2,504 under both profiles. Stage timings, resident memory, transient workspace, and retained artifact sizes are measured separately. Replicated scaling-series measurements are distinguished from single complete-VCF and corpus runs.

VCF-RDFizer vs. VCF Parsers cost comparison. Retrieval experiments compare indexed QLever queries with full VCF parsing using cyvcf2. For five regional questions, the comparison also includes tabix-indexed VCF access through cyvcf2 and bcftools. Individual-question timings measure each question separately, requiring a full VCF parse for each question on the non-indexed VCF path. Batch timings measure all 13 questions together, requiring only one full VCF parse. Regional questions select records by genomic position (POS) within windows ranging from 1 kilobase (kb) to 10 megabases (Mb), using identical selection criteria across execution paths. Setup costs are reported separately from query execution. RDF setup includes conversion and index construction, whereas VCF setup includes bgzip compression and tabix indexing.

3.3.2 Results

VCF-to-RDF converters comparison. On the shared inputs, VCF-RDFizer was the only converter whose graphs matched the source-derived oracle for all eight cross-vocabulary content questions (Figure 6). The other outputs differed because information was absent or transformed, not because different query engines were used. JVarkit merged five pairs of cohort records that shared chromosome, position, and reference allele. SPARQLing Genomics represented only calls carrying an alternate allele and therefore omitted homozygous-reference cohort calls. TogoVar represented

allele-frequency data but not samples or headers and omitted symbolic and spanning-deletion alleles. BioInterchange answered all questions on the cohort input, but invalid JSON in the HG005 header output prevented recovery of `Q7` and `Q8`.

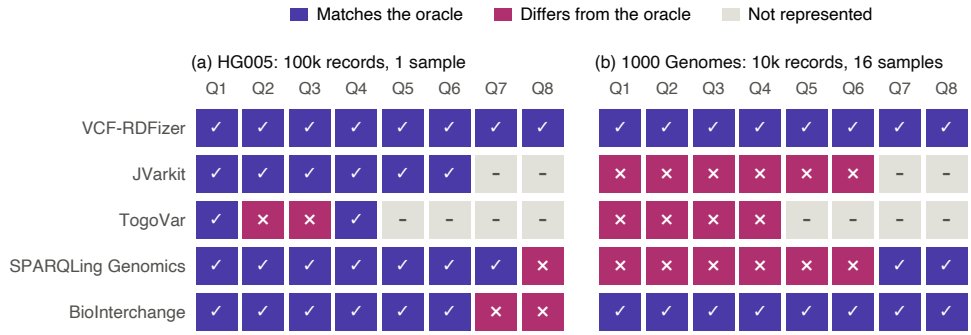


Fig. 6 Cross-vocabulary content-question results on two shared inputs. Each cell compares one ported question with the answer computed directly from the VCF: a check mark denotes agreement, a cross disagreement, and a dash information not represented in the graph. `Q1`–`Q4` summarize records, `Q5`–`Q6` genotypes, and `Q7`–`Q8` the header.

Broader coverage carried additional cost. The other converters produced 14–57 triples per record on HG005 and 18–203 on the 16-sample input, and completed each input in 0.1–6.3 seconds, whereas VCF-RDFizer produced 171 triples per HG005 record and 536 per record on the 16-sample input, and took 31–92 seconds per input, including container startup for each stage. Supplementary Section S7 reports output sizes, memory, and per-question causes.

VCF-RDFizer scaling experiments. The record-count series produced 170.5–171.9 triples per VCF record, with mapping-stage resident memory between 1.0 and 1.7 GB as record count increased. Wall time and disk use grew with the data, while the observed mapping memory remained bounded over the tested range. Sample representation, not record count, determined the number of triples per record. At 10,000 records and 2,504 samples, expanded output contained 627.4 million triples and a 4.10 GB HDT, compared with a 59.2 MB HDT for condensed output. The condensed profile reduces repeated graph structure but requires vector-aware querying and validation. Supplementary Figure S6 gives the complete sample-count series.

Optional artifacts dominated one whole-VCF benchmark. The conversion of the complete HG005 VCF required 16.07 hours when both HDT and COTTAS were built, of which approximately 14.5 hours were spent constructing these compressed representations. Retaining HDT, its index, and COTTAS required 10.88 GB, while gzip-framed N-Triples alone required 2.89 GB.

VCF-RDFizer vs. VCF Parsers cost comparison. On the 17.1-million-triple graph of 100,000 HG005 records, individual indexed QLever queries required 0.005–4.62 seconds, compared with 11.5–12.3 seconds for full cyvcf2 parses. For the 13-question batch, one cyvcf2 pass required 12.4 seconds, while the QLever queries required 17.1 seconds, excluding setup. The minimal RDF setup, which converts the

records to gzip-framed N-Triples (75.8 MB) and builds a QLever index (214.5 MB) without HDT or COTTAS, required about two minutes; under the tested repeated-question workload, the cumulative times crossed after approximately 11 separately executed questions, because each question saved about 11 seconds (a median of 12.28 seconds per full parse against a mean of 1.43 seconds per indexed query). The N-Triples were 21–25 times larger than the gzipped VCF.

Regional queries give a fairer comparison, because the VCF parsing side can also use an index and read only the requested region (Figure 7). Across 11,160 executions, all answers agreed and no executions failed. For windows up to 100 kb, both tabix-indexed VCF readers were faster than QLever, taking 3.3–4.6 ms per question with cyvcf2 and 4.2–4.6 ms with bcftools, against 10.3–10.8 ms for QLever at every window size (medians, excluding setup). At 1 Mb, bcftools remained faster (7.3 ms) while cyvcf2 became slower (14.5 ms), and at 10 Mb QLever was faster than both (31.5 ms with bcftools and 118.3 ms with cyvcf2). QLever query time remained nearly constant across window sizes, whereas VCF-region retrieval time increased with the number of records parsed. Building the QLever index from existing N-Triples required 23.3 seconds, and bgzip and tabix required 0.46 seconds. The QLever index required 60.5 times the storage of the bgzipped VCF with its tabix index (214.5 MB against 3.5 MB), and 57.5 times for the complete HG005 VCF (8.1 GB against 140.9 MB).

On the 657.4-million-triple graph of the complete HG005 VCF, QLever answered all 13 questions in 646.4 seconds after an 880.8-second index build, with every answer equal to the parser-derived result. Loading the same graph in QLever from N-Triples, HDT, or COTTAS changed indexed query time by only 0.4%, although decode costs differed. When Comunica queried the HDT and COTTAS files directly, without a QLever index, it did not complete the full workload on the 171-million-triple graph of one million HG005 records as reported in Section 3.1.2.

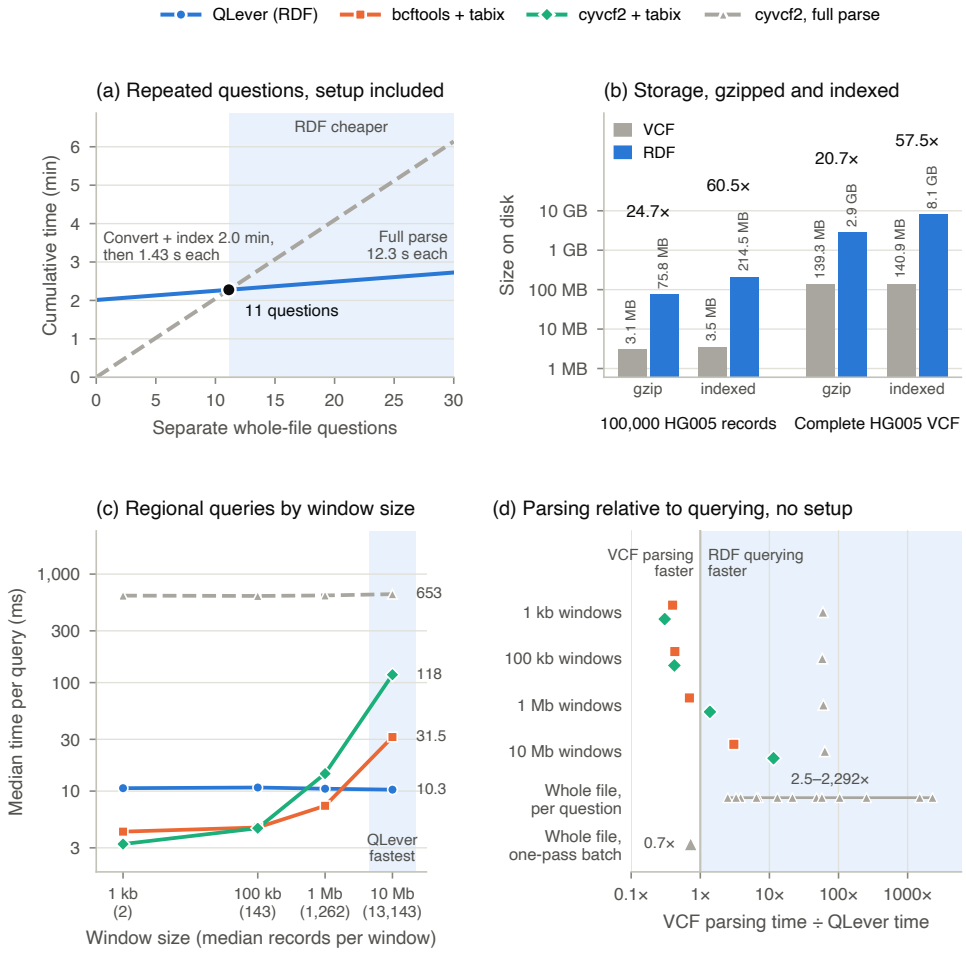


Fig. 7 Indexed RDF querying can offset conversion costs through repeated queries, while VCF parsing remains faster for narrow regions and one-pass analyses. Data: 100,000 HG005 records (17.1 million triples) unless noted; a whole-file question is one of the 13 source-comparison queries, and the five regional questions are window-restricted variants of `Q1`–`Q5`. (a) Including approximately two minutes for RDF conversion and indexing, cumulative RDF time becomes lower than that of repeated VCF scans after 11 separately executed questions. (b) Gzip-framed N-Triples require 20.7–24.7 times more storage than the corresponding gzip-compressed VCF; the QLever index requires 57.5–60.5 times the storage of the bgzipped VCF with its tabix index. (c) Regional queries show nearly constant QLever execution times as window size increases from 1 kb to 10 Mb, while indexed VCF retrieval becomes slower for larger windows. (d) Ratios of VCF parsing time to QLever query time, excluding setup, show which approach performs better across workloads; values above one favor QLever. Together, the results show that repeated and large-region queries can offset RDF's additional processing time, whereas direct VCF access remains advantageous for regions up to 100 kb and when all questions are answered in one VCF pass.

4 Discussion

This section first relates the evaluated VCF-RDFizer tool and the VCF representations it generates to the three motivating use cases (RQ2; Sections 4.1–4.4). It then discusses what the validation results establish (RQ1; Section 4.5) and when the measured costs favor RDF or direct VCF processing (RQ3; Section 4.6), and closes with the scope of the study and next steps.

4.1 A first step toward reusable genomic information

Clinical reanalysis, pharmacogenomic decision support, and consent-aware genomic reuse require more than converting VCF into another serialization. Each application depends on explicit links between source observations and information maintained elsewhere, including evidence, clinical context, and conditions governing data use. VCF-RDFizer provides an evaluated representation layer by retaining source context, validating selected properties of the RDF, and demonstrating integration and policy-aware release. The study evaluates that foundation rather than the clinical effectiveness, operational utility, or security of complete applications.

Agreement with the conventional workflow confirms that the demonstrated task is not exclusive to RDF. The distinction is representational: observations, external resources, shared identifiers, and policies are expressed in a common graph that can be queried and extended without rewriting the source representation. The separate frequency graph and identity-targeted policies demonstrate reuse within the tested workflow, but do not establish a general reduction in development effort or imply that conventional pipelines cannot provide equivalent functionality.

4.2 Clinical interpretation and reanalysis

In the evaluated workflow, each converted VCF was joined to a fixed ClinVar release through SPDI identifiers, and the workflow returned the same matches as the conventional workflow in every scenario. RDF made the evidence behind each match explicit and inspectable: in Scenario 2, ClinVar classifications and panel frequencies were joined from separate graphs rather than copied into the VCF (Section 3.2, Figure 5a). Separate linksets allow new evidence and its provenance to be linked to genomic observations without modifying the converted VCF or replacing earlier assertions (Section 2.3). Building on Scenario 2's join to separately stored frequencies, the same approach could identify observations affected by revised ClinVar classifications or frequency estimates [8, 9]. Such queries could support clinical reanalysis without replacing expert interpretation or recovering variants absent from the original call set.

The evaluated ClinVar and frequency joins provide a starting point for longitudinal reanalysis with VCF-RDFizer's linking model. Because linksets are stored separately from the converted graph, changing clinical assertions, disease context, review status, provenance, and dates could be added as they change and connected to patient phenotypes, family information, and prior reports. A change-detection process could then compare successive linksets and route affected observations for clinical review. Such joins depend on equivalent alleles receiving the same identifier. The minimally trimmed SPDI expressions used here keep the position written in the VCF, which

preserves traceability to the source record, but in repetitive regions they can differ from NCBI's contextual SPDI, so equivalent alleles can miss evidence joins [25]. For example, a one-base deletion in a run of identical bases can be written at any position in the run, whereas the contextual SPDI spans the whole run and gives every such description the same identifier. Adding contextual SPDI as a further linkset would strengthen these joins without changing the converted data. Future evaluations could compare the cases nominated through these links with conventional reannotation and expert-reviewed outcomes, including missed updates and unnecessary reviews. The static query agreement measured here does not yet demonstrate clinical value, but it provides a verified, linked foundation on which future work could pursue the benefits reported in reanalysis studies [10].

4.3 Pharmacogenomic clinical decision support

The evaluated linkers connected variant calls to overlapping genes and alleles to shared identifiers, the kind of link a pharmacogenomic decision-support path requires, although no pharmacogenomic interpretation was evaluated. Extending this path to a pharmacogenomic interpretation, medication order, indication, and prescribing guidance would add further linksets of the kind evaluated in Scenario 2 (Section 3.2). As with the frequency graph, stored once and joined to every participant in Scenario 2 (Section 3.2.1), separate linked resources could preserve which observation, interpretation, and guideline version support a recommendation and allow an existing interpretation to be reused when the prescribing context changes. The model addresses an integration challenge reported in pharmacogenomic decision-support research without replacing established clinical systems or interpretation tools [12, 13].

The present workflow demonstrates links between genomic observations and external knowledge but does not infer diplotypes, metabolizer phenotypes, or treatment recommendations. A pharmacogenomic extension could connect represented calls to a validated interpretation pipeline, such as PharmCAT, while retaining interpretation assumptions, supporting evidence, and unresolved results [2]. Reliable interpretation requires adequate variant coverage and phasing, as the absence of a VCF record does not establish a reference genotype. The resulting interpretation could then be linked to the patient, medication, and clinical indication through shared identifiers and exchange models, including FHIR Genomics [42]. For example, CYP2C19-guided clopidogrel recommendations depend on both metabolizer phenotype and clinical indication [11]. With versioned guidance, explicit handling of missing or conflicting data, and expert validation, this linking model could support traceable pharmacogenomic decision support [12].

4.4 Consent-aware genomic reuse

Expressing consent as data rather than code changes where release decisions can be examined. Because rules referred directly to files, linked genes, regions, and variant identities, we showed that one generic evaluator can serve every scenario, and a new restriction became a policy edit rather than a code change (Section 3.2). Recording the policy and request with each view makes every release decision auditable after the

fact, and agreement with an independently written conventional workflow indicates that the evaluator applied the intended rules rather than merely producing output.

The limitations are equally instructive. The Scenario 2 leak shows that a rule is only as complete as the annotations it selects by, and that a release check sharing those annotations inherits their gaps. Gene-based rules over structural variants therefore remain unreliable in our implementation until symbolic alleles are linked by their extent. Another limitation encountered is the cost of policy evaluation. The computational cost comes from materializing a full view for every requester, so it grows with both graph size and the number of requesters, reaching about an hour per view on a 1.2-billion-triple graph. Our approach therefore suits a few stable views better than frequent or ad hoc requests. Selective masking within multi-sample VCFs and region rules that do not depend on contig labels also remain unaddressed, and default-deny and deny-overrides are choices of this implementation rather than guarantees of RDF or ODRL [28].

Granular and verifiable policy protection of genomic data is therefore achievable, but it needs further work in three areas. Selection by linked annotations needs complete links and release checks that do not rely on them. Policy-aware SPARQL querying could avoid materializing a separate release view for every requester, while incremental view maintenance could reduce the cost of updating existing views as the underlying RDF changes [78, 79]. Both approaches remain to be evaluated with VCF-RDFizer's record-level policies. Deployment needs reviewed mappings from consent vocabularies such as DUO and the GA4GH Machine Readable Consent Guidance to an agreed policy profile, together with authenticated requesters, an enforcement boundary, and auditable versions of policies and decisions [7, 14, 15, 29].

4.5 Validation requires both source and graph checks

Our validation results show that comparing a graph with its source and checking the graph's own structure catch different kinds of errors. Source-comparison queries test what the VCF states, so they detect changed values and miscounted records, but because they summarize content they cannot tell whether a genotype points to the right allele or a header attribute belongs to the right declaration. The deeper SHACL profiles test exactly these relationships, which is why they detected every injected fault the queries missed (Section 3.1). The defects found during development tell the same story, as `Q9` and `Q10` found missing sample resources, whereas dangling allele references were missed by the queries and found only by the shapes.

The second defect also exposes the main weakness of source comparison, which is that an oracle can share the converter's assumptions. Defective `Q9` and `Q10` queries passed because their expected inventories followed the same emitter rule that produced the error, so an oracle is only as independent as the knowledge it is built from. The structural checks have a different weakness. The profiles that catch relationship errors load the whole graph into memory and have only run successfully on small fault-injection graphs, so their protection does not yet extend to complete VCF RDF representations, and their complete detection result applies to a constructed fault set. Because both deeper profiles consist entirely of SPARQL-based constraints, they could instead be evaluated as queries against an indexed SPARQL engine such

as QLever, which the evaluation already uses for complete-VCF queries, following published approaches that validate SHACL over a SPARQL endpoint rather than in memory [80, 81]. More generally, each passing check establishes only the property it tests, and none establishes the biological correctness of the original calls or exhaustive equivalence between VCF and RDF.

These findings argue for layered validation rather than any single test. Inexpensive source comparisons can run at every scale, structural checks can confirm relationships wherever they are affordable, and each layer catches errors the other misses. Future work should make relationship checks for allele links, phasing, sample indexes, and header attributes scale to complete genomes, build oracles from knowledge independent of the converter's own rules, and compare values in ways that tolerate permitted lexical differences.

4.6 Representation and workload determine the deployment choice

RDF is an investment that pays off only for certain workloads. Direct VCF parsing remains the better choice for one-pass analyses and small regions because it requires no conversion or indexing. When all 13 questions were known in advance, a single cyvcf2 pass answered them faster than QLever, even before counting RDF setup. RDF became cheaper only when questions were asked separately and repeatedly, after about 11 questions in the tested workload, or when large regions were queried, because QLever's query time stayed nearly constant as windows grew. This crossover reflects the measured workload and setup, not a fixed property of the tool.

Representation choices shift the same balance. The expanded profile keeps every sample call directly addressable, which sample-level queries, validation, and policies require, whereas the condensed profile reduces graph size by storing sample values in vectors at the cost of vector-aware querying. Optional HDT and COTTAS artifacts dominated whole-VCF conversion time and are worth generating only when their storage or access properties are needed. Triple count alone therefore does not indicate which representation suits an application.

Together, these results suggest that RDF suits genomic data that are queried repeatedly, linked to independently maintained resources, or released under changing policies, as in the RQ2 workflow, rather than one-off analyses. A reanalysis service, for example, could keep one converted VCF and query successive evidence releases against it. The cost of updating links, indexes, and release views as evidence or policies change was not measured and is the next quantity to establish. For one-off or exploratory use cases, combining direct VCF parsing with RDF virtualization, which answers SPARQL queries over the source data without materializing a graph, could balance the advantages of both approaches [82].

4.7 VCF-RDFizer in the VCF-to-RDF converter landscape

We show that existing tools and VCF-RDFizer optimize for different goals. The other converters produce compact graphs quickly, but each omits or transforms some VCF content, such as homozygous-reference calls, samples and headers, symbolic alleles, or

distinct records that share a position (Section 3.3). Some of these differences reflect narrower design goals, such as TogoVar's focus on allele frequencies, while others, such as merged records and invalid header output, are conversion errors. Either way, they limit questions that need the complete record, including the genotype, header, and source context on which reanalysis and governed release depend. VCF-RDFizer was the only converter whose graphs answered all eight questions correctly on both inputs. BioInterchange came closest, answering all eight questions on the cohort input and six of eight on HG005, where invalid JSON in its header output, a serialization error rather than a design omission, prevented recovery of the two header questions. The eight assessed questions cover record positions, allele classes, filters, genotypes, allele counts, and header metadata, but not individual INFO and FORMAT values, phasing, or structural-variant detail, which `Q9`–`Q13` assess only in VCF-RDFizer's own vocabulary. A more thorough comparison should therefore port value-level checks to each converter's vocabulary and add more diverse inputs, such as larger cohorts and structural-variant call sets.

The comparison also shows that the validation approach is not specific to VCF-RDFizer. The same source-derived oracle, ported to each vocabulary, located where every converter lost information, so source comparison could serve as a common benchmark for VCF-to-RDF conversion.

4.8 Scope and next steps

The shared-input converter comparison covers four converters, two inputs, and `Q1`–`Q8`. The result establishes comparative content coverage under the tested configurations, not universal superiority or suitability for every purpose. The sample-count series held INFO content fixed, the complete multi-sample cohort was not converted, and the deeper SHACL profiles were evaluated only on small inputs. The linking experiments use single-sample files and simulated policies, and all outcomes remain tied to the recorded data, software, reference releases, and policy configurations. Gene-based policy selection also remains incomplete for some symbolic and breakend records, as documented in Supplementary Section S10.

Future work can proceed in stages. The first priority is to close remaining oracle gaps and define stronger boundaries for normalization and shared variant identity. Application-specific pilots can then evaluate versioned evidence updates, validated pharmacogenomic interpretations linked to clinical context, and selective release under reviewed policy models. Each pilot should involve the relevant domain experts and an application-specific comparator. Clinical deployment would additionally require governance, privacy, security, and human-workflow assessments appropriate to the intended environment. GA4GH VRS identifiers, federated querying, and policy evaluation over multi-sample data remain promising extensions.

VCF-RDFizer provides a testable starting point for this progression: a genomic representation that can be validated against its source, linked to external information, and reused across workflows while the additional requirements of each application remain explicit.

5 Conclusions

VCF-RDFizer makes VCF information explicit in a representation that can be validated against its VCF input, linked to external knowledge, and used in policy-aware research queries. The evaluation demonstrates complementary validation, reusable joins, and requester-specific agreement with a conventional workflow, while showing that semantic coverage adds conversion, storage, and indexing costs. The resulting representation is therefore most relevant when persistent links, repeated questions, or inspectable release rules justify those costs, rather than as a universal replacement for direct VCF processing. VCF-RDFizer is a first step toward evidence-aware reanalysis, pharmacogenomic decision support, and consent-aware reuse; complete applications still require validated interpretation, clinical integration, and enforced access processes.

Availability and requirements

Project: VCF-RDFizer. **Home page:** https://github.com/ecrum19/VCF-RDFizer. **Language:** Python, with the mapping, compression, and query tools specified in Supplementary Section S3. **Platforms:** Linux, macOS, and Windows are covered by the tool's tests; the experiments reported here ran on Ubuntu. **Distribution:** PyPI, conda-forge, and Docker. **License:** MIT. **Any restrictions to use by non-academics:** none. Docker supplies the pinned benchmark environment; requirements for other installations are documented with each release. Version 3.1.0 produced the core benchmark results and the N-Triples-only and fault-injection reruns, and v3.3.1 produced those of the linked workflow, regional and large-graph retrieval, the index-storage measurement, the consumer WGS validation run, and the converter comparison. On the same 100,000 HG005 records, both releases wrote identical sorted triples. Supplementary Section S3.2 records the build behind each result.

Declarations

Ethics approval and consent to participate

This study used publicly released genomic data and involved no recruitment of or interaction with participants. Personal Genome Project donors released their files under the project's open-consent model [65]; GIAB and precisionFDA files are public reference materials [66, 83]; the 1000 Genomes and HGSVC2 datasets are released for research reuse [68, 69, 76]; and ClinVar is a public NCBI resource [77]. The experimental consents are simulated and do not express any real participant's wishes. No identification of participants was attempted. Ethics approval was not sought for the reported work.

Consent for publication

Not applicable. The manuscript reports aggregate results and software measurements, not individual-level genomic records, images, or personal details.

Availability of data and materials

Input filenames, provider URLs, download commands, fixtures, derivation scripts, query files, manifests, normalized outputs, and comparison reports are maintained at https://github.com/ecrum19/vcf-rdfizer-testing. The benchmark harness and the run records behind every reported result are archived on Zenodo at https://doi.org/10.5281/zenodo.23301185; version v1.0.0, which holds the results reported here, is https://doi.org/10.5281/zenodo.23301186 [84]. Large source VCFs are obtained from their public providers rather than redistributed in Git. Experiment 17 (`benchmarks/use_case/acmg`) contains the use-case definitions, simulated policies, conventional baseline, and applied-change tests. Its match sets, timings, and check reports, as well as the additional validation runs, are recorded under `benchmark-results`. The main branch holds the runs behind the reported results; pre-release, superseded, and failed runs, including those cited as defect evidence in the Supplementary Material, are archived on the repository's `legacy` branch. No biological materials were generated or used.

The software is available at the project home page. The core benchmark and the N-Triples-only and fault-injection reruns used v3.1.0 (`d3b34d5`; image `sha256:1904e96d...34aa`). The linked workflow, regional and large-graph retrieval, index-storage measurement, converter comparison, and consumer WGS validation run used v3.3.1 (`b25fb7b`; image `sha256:3ad71b1a...2993`), a release that includes the evaluated linkers and the policy plug-in. Supplementary Section S3 identifies each experiment's build, the image digests, and the bundled shape snapshot. The public vocabulary is separately versioned [46]. Results must be traced to their recorded build and input, not only to the latest package.

Competing interests

None declared.

Funding

Elias Crum acknowledges support from the Research Foundation–Flanders (FWO), SB Fellowship 1S27825N. Ruben Taelman is a postdoctoral fellow of the Research Foundation – Flanders (FWO) (1202124N). Gokhan Ertaylan acknowledges support from the European Union under the Horizon Europe programme through the REALM project (grant agreement No. 101095435), the European Union's Horizon Europe research and innovation programme through the RAIDO (Reliable AI and Data Optimisation) project under grant agreement No. 101135800, and the Belgian Federal Public Service Policy & Support (BOSA) through the DigiTwin4PH (Digital Twins for Personalised Healthcare) project. Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the granting authorities.

Authors' contributions

E.C., R.T., and G.E. conceived the study. E.C. did the implementation and coding work. E.C. devised the validation and use-case demonstration with the guidance of

R.T. and executed the experiments. E.C., R.T., and G.E. reviewed and verified all results. E.C., B.B., R.T., and G.E. wrote and edited the manuscript. R.T. and G.E. supervised this work.

Acknowledgments

Benchmarking used the resources of IDLab at Ghent University. Claude Opus 5.5 [85] and GPT-6 Astra [86] assisted with experiment automation scripting, result data analysis scripting, and editing of this manuscript. The authors reviewed all AI outputs and take full responsibility for the manuscript, experiments, tools, and repositories.

References

[1] SAM/BAM Format Specification Working Group: The Variant Call Format Specification (2024). https://github.com/samtools/hts-specs

[2] Li, B., Sangkuhl, K., Keat, K., Whaley, R.M., Woon, M., Verma, S., Dudek, S., Tuteja, S., Verma, A., Whirl-Carrillo, M., Ritchie, M.D., Klein, T.E.: How to run the pharmacogenomics clinical annotation tool (pharmcat). Clinical Pharmacology & Therapeutics **113**(5), 1036–1047 (2023) https://doi.org/10.1002/cpt.2790

[3] Souche, E., Beltran, S., Brosens, E., Belmont, J.W., Fossum, M., Riess, O., Gilissen, C., Ardeshirdavani, A., Houge, G., Van Gijn, M., Clayton-Smith, J., Synofzik, M., De Leeuw, N., Deans, Z.C., Dincer, Y., Eck, S.H., Van Der Crabben, S., Balasubramanian, M., Graessner, H., Sturm, M., Firth, H., Ferlini, A., Nabbout, R., De Baere, E., Liehr, T., Macek, M., Matthijs, G., Scheffer, H., Bauer, P., Yntema, H.G., Weiss, M.M.: Recommendations for whole genome sequencing in diagnostics for rare diseases. European Journal of Human Genetics **30**(9), 1017–1021 (2022) https://doi.org/10.1038/s41431-022-01113-x

[4] Danecek, P., Bonfield, J.K., Liddle, J., Marshall, J., Ohan, V., Pollard, M.O., Whitwham, A., Keane, T., McCarthy, S.A., Davies, R.M., Li, H.: Twelve years of samtools and bcftools. GigaScience **10**(2), 008 (2021) https://doi.org/10.1093/gigascience/giab008

[5] Gottesman, O., Kuivaniemi, H., Tromp, G., Faucett, W.A., Li, R., Manolio, T.A., *et al.*: The Electronic Medical Records and Genomics (eMERGE) Network: past, present, and future. Genetics in Medicine **15**(10), 761–771 (2013) https://doi.org/10.1038/gim.2013.72

[6] The All of Us Research Program Genomics Investigators: Genomic data in the All of Us research program. Nature **627**(8003), 340–346 (2024) https://doi.org/10.1038/s41586-023-06957-x

[7] Rehm, H.L., *et al.*: GA4GH: International policies and standards for data sharing across genomic research and healthcare. Cell Genomics **1**(2), 100029 (2021) https://doi.org/10.1016/j.xgen.2021.100029

[8] Richards, S., Aziz, N., Bale, S., Bick, D., Das, S., *et al.*: Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. Genetics in Medicine **17**(5), 405–424 (2015) https://doi.org/10.1038/gim.2015.30

[9] Deignan, J.L., Chung, W.K., Kearney, H.M., Monaghan, K.G., Rehder, C.W., Chao, E.C.: Points to consider in the reevaluation and reanalysis of genomic test results: a statement of the American College of Medical Genetics and Genomics

(ACMG). Genetics in Medicine **21**(6), 1267–1270 (2019) https://doi.org/10.1038/s41436-019-0478-1

[10] Hiatt, S.M., Amaral, M.D., Bowling, K.M., Finnila, C.R., Thompson, M.L., Gray, D.E., Lawlor, J.M.J., Cochran, J.N., Bebin, E.M., Brothers, K.B., East, K.M., Kelley, W.V., Lamb, N.E., Levy, S.E., Lose, E.J., Neu, M.B., Rich, C.A., Simmons, S., Myers, R.M., Barsh, G.S., Cooper, G.M.: Systematic reanalysis of genomic data improves quality of variant interpretation. Clinical Genetics **94**(1), 174–178 (2018) https://doi.org/10.1111/cge.13259

[11] Lee, C.R., Luzum, J.A., Sangkuhl, K., Gammal, R.S., Sabatine, M.S., Stein, C.M., Kisor, D.F., Limdi, N.A., Lee, Y.M., Scott, S.A., Hulot, J.-S., Roden, D.M., Gaedigk, A., Caudle, K.E., Klein, T.E., Johnson, J.A., Shuldiner, A.R.: Clinical pharmacogenetics implementation consortium guideline for CYP2C19 genotype and clopidogrel therapy: 2022 update. Clinical Pharmacology & Therapeutics **112**(5), 959–967 (2022) https://doi.org/10.1002/cpt.2526

[12] Wake, D.T., Smith, D.M., Kazi, S., Dunnenberger, H.M.: Pharmacogenomic clinical decision support: A review, how-to guide, and future vision. Clinical Pharmacology & Therapeutics **112**(1), 44–57 (2022) https://doi.org/10.1002/cpt.2387

[13] Johnson, D., Del Fiol, G., Kawamoto, K., Romagnoli, K.M., Sanders, N., Isaacson, G., Jenkins, E., Williams, M.S.: Genetically guided precision medicine clinical decision support tools: a systematic review. Journal of the American Medical Informatics Association **31**(5), 1183–1194 (2024) https://doi.org/10.1093/jamia/ocae033

[14] Lawson, J., Cabili, M.N., Kerry, G., *et al.*: The Data Use Ontology to streamline responsible access to human biomedical datasets. Cell Genomics **1**(2), 100028 (2021) https://doi.org/10.1016/j.xgen.2021.100028

[15] Global Alliance for Genomics and Health: Machine Readable Consent Guidance (MRCG). Technical implementation guidance, version 1.0. Accessed 7 October 2026. https://www.ga4gh.org/product/machine-readable-consent-guidance/ Accessed 2026-10-07

[16] European Parliament and Council of the European Union: Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation). Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88 (2016). https://eur-lex.europa.eu/eli/reg/2016/679/oj

[17] Shabani, M., Borry, P.: Rules for processing genetic data for research purposes in view of the new EU General Data Protection Regulation. European Journal of Human Genetics **26**(2), 149–156 (2018) https://doi.org/10.1038/s41431-017

-0045-7

[18] Paila, U., Chapman, B.A., Kirchner, R., Quinlan, A.R.: GEMINI: integrative exploration of genetic variation and genome annotations. PLoS Computational Biology **9**(7), 1003153 (2013) https://doi.org/10.1371/journal.pcbi.1003153

[19] Danecek, P., Auton, A., Abecasis, G., Albers, C.A., Banks, E., DePristo, M.A., Handsaker, R.E., Lunter, G., Marth, G.T., Sherry, S.T., McVean, G., Durbin, R., 1000 Genomes Project Analysis Group: The variant call format and VCFtools. Bioinformatics **27**(15), 2156–2158 (2011) https://doi.org/10.1093/bioinformatics/btr330

[20] Cyganiak, R., Wood, D., Lanthaler, M.: RDF 1.1: Concepts and Abstract Syntax (2014). https://www.w3.org/TR/2014/REC-rdf11-concepts-20140225/

[21] Hogan, A., Blomqvist, E., Cochez, M., D'Amato, C., Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.-C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A.: Knowledge graphs. ACM Computing Surveys **54**(4) (2021) https://doi.org/10.1145/3447772

[22] World Wide Web Consortium (W3C): LinkedData. https://www.w3.org/wiki/LinkedData Accessed 2026-03-04

[23] Harris, S., Seaborne, A., Prud'hommeaux, E.: SPARQL 1.1 Query Language (2013). https://www.w3.org/TR/2013/REC-sparql11-query-20130321/

[24] Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J.G., Groth, P., Goble, C., Grethe, J.S., Heringa, J., Hoen, P.A.C., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., Schaik, R., Sansone, S.-A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M.A., Thompson, M., Lei, J., Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B.: The fair guiding principles for scientific data management and stewardship. Scientific Data **3**, 160018 (2016) https://doi.org/10.1038/sdata.2016.18

[25] Holmes, J.B., Moyer, E., Phan, L., Maglott, D., Kattman, B.: SPDI: data model for variants and applications at NCBI. Bioinformatics **36**(6), 1902–1907 (2020) https://doi.org/10.1093/bioinformatics/btz856

[26] Sherry, S.T., Ward, M., Kholodov, M., Baker, J., Phan, L., Smigielski, E.M., Sirotkin, K.: dbsnp: the ncbi database of genetic variation. Nucleic Acids Research **29**(1), 308–311 (2001) https://doi.org/10.1093/nar/29.1.308

[27] Landrum, M.J., Lee, J.M., Benson, M., Brown, G.R., Chao, C., Chitipiralla, S., Gu, B., Hart, J., Hoffman, D., Jang, W., Karapetyan, K., Katz, K., Liu, C., Maddipatla, Z., Malheiro, A., McDaniel, K., Ovetsky, M., Riley, G., Zhou, G., Holmes, J.B., Kattman, B.L., Maglott, D.R.: ClinVar: improving access to variant interpretations and supporting evidence. Nucleic Acids Research **46**(D1), 1062–1067 (2018) https://doi.org/10.1093/nar/gkx1153

[28] Iannella, R., Villata, S.: ODRL Information Model 2.2. W3C (2018). https://www.w3.org/TR/odrl-model/ Accessed 2025-11-27

[29] OASIS: eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard (2013). https://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html

[30] Katayama, T., Kawashima, S., Micklem, G., Kawano, S., Kim, J.-D., Kocbek, S., Okamoto, S., Wang, Y., Wu, H., Yamaguchi, A., *et al.*: Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. F1000Research **8**, 1677 (2019) https://doi.org/10.12688/f1000research.18238.1

[31] Lindenbaum, P.: JVarkit: java-based utilities for Bioinformatics. figshare (2015). https://doi.org/10.6084/m9.figshare.1425030.v1 . https://figshare.com/articles/journal_contribution/JVarkit_java_based_utilities_for_Bioinformatics/1425030

[32] Garrison, E., Kronenberg, Z.N., Dawson, E.T., Pedersen, B.S., Prins, P.: A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. PLOS Computational Biology **18**(5), 1009123 (2022) https://doi.org/10.1371/journal.pcbi.1009123

[33] Mitsuhashi, N., Toyo-oka, L., Katayama, T., Kawashima, M., Kawashima, S., Miyazaki, K., Takagi, T.: TogoVar: A comprehensive Japanese genetic variation database. Human Genome Variation **9**(1), 44 (2022) https://doi.org/10.1038/s41439-022-00222-9

[34] Bodrug-Schepers, A., Chabane, H., Montoya, G., Serrano-Alvarado, P., Redon, R., Gaignard, A.: Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In: Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025). CEUR Workshop Proceedings, vol. 4196. CEUR Workshop Proceedings, Barcelona, Spain (2025). https://ceur-ws.org/Vol-4196/paper_16.pdf Accessed 2025-11-24

[35] Böhmer, J.K., Janssen, R.: Linked data in practice: An RDF-based approach with SPARQLing Genomics. Zenodo. Zenodo record, version 4 (2020). https://doi.org/10.5281/zenodo.3903111

[36] Janssen, R.: SPARQLing Genomics. Open-source software. Accessed 2026-09-03

(2019). https://gitlab.com/roelj/sparqling-genomics

[37] Baran, J., Durgahee, B.S.B., Eilbeck, K., Antezana, E., Hoehndorf, R., Dumontier, M.: GFVO: the Genomic Feature and Variation Ontology. PeerJ **3**, 933 (2015) https://doi.org/10.7717/peerj.933

[38] Hail Team: Hail: scalable genomic data analysis. Accessed October 2026 (2026). https://github.com/hail-is/hail

[39] TileDB, Inc.: TileDB-VCF: efficient variant-call data storage and retrieval. Accessed October 2026 (2026). https://github.com/TileDB-Inc/TileDB-VCF

[40] Rambla, J., Baudis, M., Ariosa, R., *et al.*: Beacon v2 and Beacon networks: A "lingua franca" for federated data discovery in biomedical genomics, and beyond. Human Mutation **43**(6), 791–799 (2022) https://doi.org/10.1002/humu.24369

[41] Kelleher, J., Lin, M., Albach, C.H., Birney, E., *et al.*: htsget: a protocol for securely streaming genomic data. Bioinformatics **35**(1), 119–121 (2019) https://doi.org/10.1093/bioinformatics/bty492

[42] HL7 International: HL7 FHIR Genomics Reporting Implementation Guide. Accessed October 2026 (2026). https://hl7.org/fhir/uv/genomics-reporting/

[43] Touré, V., Unni, D., Krauss, P., Abdelwahed, A., Buchhorn, J., Hinderling, L., Geiger, T.R., Österle, S.: The SPHN Schema Forge – transform healthcare semantics from human-readable to machine-readable by leveraging semantic web technologies. Journal of Biomedical Semantics **16**, 9 (2025) https://doi.org/10.1186/s13326-025-00330-9

[44] Gaignard, A., Rosnet, T., Lamotte, F., Lefort, V., Devignes, M.-D.: FAIR-Checker: supporting digital resource findability and reuse with knowledge graphs and semantic web standards. Journal of Biomedical Semantics **14**, 7 (2023) https://doi.org/10.1186/s13326-023-00289-5

[45] Ansari, A., Conte, M., Flynn, A., Paturkar, A.: A prototype ETL pipeline that uses HL7 FHIR RDF resources when deploying pure functions to enrich knowledge graph patient data. Journal of Biomedical Semantics **16**, 16 (2025) https://doi.org/10.1186/s13326-025-00335-4

[46] Crum, E.: VCF Core Vocabulary. Version 2.1.3. GitHub repository. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026 (2026). https://github.com/ecrum19/vcf-core-vocabulary/tree/8a5f23a

[47] Crum, E., Buelens, B., Ertaylan, G., Taelman, R.: VCF Core Vocabulary: A Semantic Foundation for RDF Representations of VCF. Manuscript in preparation for SWAT4HCLS 2027; preprint at commit 9507dd9 (2026). https://github.com/ecrum19/SWAT4HCLS_2027/blob/9507dd9/long_revised/main_long.pdf

[48] VCF-RDFizer maintainers: ecrum19/vcf-rdfizer. Docker Hub image, tag 3.3.1, digest sha256:3ad71b1a54612142e3be43b2 4e7e4a38551949cb641f4421bcae9bf051102993. Published 2026-10-08 (2026). https://hub.docker.com/r/ecrum19/vcf-rdfizer

[49] VCF-RDFizer maintainers: vcf-rdfizer. Python Package Index (PyPI), version 3.3.1. Released 2026-10-08 (2026). https://pypi.org/project/vcf-rdfizer/3.3.1/

[50] VCF-RDFizer maintainers: vcf-rdfizer. conda-forge, version 3.3.1. Published 2026-10-08 (2026). https://anaconda.org/conda-forge/vcf-rdfizer

[51] Bolleman, J.T., Mungall, C.J., Strozzi, F., Baran, J., Dumontier, M., Bonnal, R.J.P., Buels, R., Hoehndorf, R., Fujisawa, T., Katayama, T., Cock, P.J.A.: FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. Journal of Biomedical Semantics **7**(1), 39 (2016) https://doi.org/10.1186/s13326-016-0067-z

[52] Eilbeck, K., Lewis, S.E., Mungall, C.J., Yandell, M., Stein, L., Durbin, R., Ashburner, M.: The Sequence Ontology: a tool for the unification of genome annotations. Genome Biology **6**(5), 44 (2005) https://doi.org/10.1186/gb-2005-6-5-r44

[53] Menotti, L., Cazzaro, M., Rueda, M., Gut, I.G., Silvello, G.: HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In: Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025). CEUR Workshop Proceedings, Barcelona, Spain (2025). https://www.dei.unipd.it/~silvello/papers/2025_MCS_SWAT4HCLS.pdf

[54] Wagner, A.H., Babb, L., Alterovitz, G., Baudis, M., Brush, M., Cameron, D.L., Cline, M., Griffith, M., Griffith, O.L., Hunt, S.E., Kreda, D., Lee, J.M., Li, S., Lopez, J., Moyer, E., Nelson, T., Patel, R.Y., Riehle, K., Robinson, P.N., Rynearson, S., Schuilenburg, H., Tsukanov, K., Walsh, B., Konopko, M., Rehm, H.L., Yates, A.D., Freimuth, R.R., Hart, R.K.: The GA4GH Variation Representation Specification: A computational framework for variation representation and federated identification. Cell Genomics **1**(2), 100027 (2021) https://doi.org/10.1016/j.xgen.2021.100027

[55] Wagner, A.H., Hart, R., Babb, L., Kuzma, K., Schuilenburg, H., Stevenson, J., ONeill, T., Epic, M.-T., Holmes, B., Moyer, E., Baudis, M., Robinson, P., Cline, M., Patel, R.Y., Freimuth, R.R.: ga4gh/vrs 2.0.1. Zenodo (2025). https://doi.org/10.5281/zenodo.15059674 . https://doi.org/10.5281/zenodo.15059674 Accessed 2026-03-02

[56] De Meester, B., Heyvaert, P., Delva, T.: RDF Mapping Language (RML) (2024). https://rml.io/specs/rml/ Accessed 2025-11-24

[57] Oo, S.M., Haesendonck, G., De Meester, B., Dimou, A.: RMLStreamer-SISO:

An RDF Stream Generator from Streaming Heterogeneous Data. In: Sattler, U., Hogan, A., Keet, M., Presutti, V., Almeida, J.P.A., Takeda, H., Monnin, P., Pirrò, G., d'Amato, C. (eds.) The Semantic Web – ISWC 2022 vol. 13489, pp. 697–713. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_40 . Series Title: Lecture Notes in Computer Science. https://link.springer.com/10.1007/978-3-031-19433-7_40 Accessed 2025-11-24

[58] Fernández, J.D., Martínez-Prieto, M.A., Gutiérrez, C., Polleres, A., Arias, M.: Binary rdf representation for publication and exchange (hdt). Web Semantics: Science, Services and Agents on the World Wide Web **19**, 22–41 (2013)

[59] Arenas-Guerrero, J., Ferrada, S.: COTTAS: Columnar triple table storage for efficient and compressed RDF management. In: The Semantic Web – ISWC 2025. Lecture Notes in Computer Science, vol. 16141, pp. 313–331. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-09530-5_18 . https://doi.org/10.1007/978-3-032-09530-5_18

[60] Alakuijala, J., Farruggia, A., Ferragina, P., Kliuchnikov, E., Obryk, R., Szabadka, Z., Vandevenne, L.: Brotli: A General-Purpose Data Compressor. ACM Transactions on Information Systems **37**(1), 1–30 (2019) https://doi.org/10.1145/3231935

[61] Pedersen, B.S., Quinlan, A.R.: cyvcf2: fast, flexible variant analysis with Python. Bioinformatics **33**(12), 1867–1869 (2017) https://doi.org/10.1093/bioinformatics/btx057

[62] World Wide Web Consortium: Shapes Constraint Language (SHACL). W3C Recommendation (2017). https://www.w3.org/TR/shacl/ Accessed 2026-10-02

[63] Garfinkel, S.L., Guttman, B., Near, J.P., Dajani, A.N., Singer, P.: De-identifying government datasets: Techniques and governance. NIST Special Publication 800-188, National Institute of Standards and Technology (2023). https://doi.org/10.6028/NIST.SP.800-188

[64] Global Alliance for Genomics and Health: Data Security Infrastructure Policy (DSIP), Version 4.0. 25 September 2019 (2019). https://github.com/ga4gh/data-security/blob/master/DSIP/DSIP_v4.0.md

[65] Personal Genome Project (2005). https://my.pgp-hms.org/public_genetic_data Accessed 2025-11-24

[66] Wagner, J., Olson, N.D., Harris, L., Khan, Z., Farek, J., Mahmoud, M., Stankovic, A., Kovacevic, V., Yoo, B., Miller, N., *et al.*: Benchmarking challenging small variants with linked and long reads. Cell Genomics **2**(5), 100128 (2022) https://doi.org/10.1016/j.xgen.2022.100128

[67] Olson, N.D., Wagner, J., McDaniel, J., *et al.*: Precisionfda truth challenge v2:

Calling variants from short and long reads in difficult-to-map regions. Cell Genomics **2**(5), 100129 (2022) https://doi.org/10.1016/j.xgen.2022.100129

[68] The 1000 Genomes Project Consortium: A global reference for human genetic variation. Nature **526**(7571), 68–74 (2015) https://doi.org/10.1038/nature15393

[69] Ebert, P., Audano, P.A., Zhu, Q., Rodriguez-Martin, B., Porubsky, D., Bonder, M.J., Sulovari, A., Ebler, J., Zhou, W., Serra Mari, R., *et al.*: Haplotype-resolved diverse human genomes and integrated analysis of structural variation. Science **372**(6537), 7117 (2021) https://doi.org/10.1126/science.abf7117

[70] Bast, H., Buchhold, B.: QLever: A query engine for efficient SPARQL+Text search. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM), pp. 647–656 (2017). https://doi.org/10.1145/3132847.3132921 . https://doi.org/10.1145/3132847.3132921

[71] Taelman, R., Herwegen, J.V., Sande, M.V., Verborgh, R.: Comunica: A modular SPARQL query engine for the web. In: Proceedings of the 17th International Semantic Web Conference, pp. 239–255. Springer, Cham (2018). https://doi.org/10.1007/978-3-030-00668-6_15 . https://comunica.dev/

[72] Comunica Association: Querying local files from the command line (2026). https://comunica.dev/docs/query/getting_started/query_cli_file/ Accessed 2026-08-31

[73] Comunica Association: HDT: Querying HDT files with Comunica (2026). https://comunica.dev/docs/query/advanced/hdt/ Accessed 2026-08-31

[74] Crum, E.: Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas` (2026). https://github.com/ecrum19/comunica-feature-cottas

[75] Miller, D.T., Lee, K., Abul-Husn, N.S., Amendola, L.M., *et al.*: ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). Genetics in Medicine **25**(8), 100866 (2023) https://doi.org/10.1016/j.gim.2023.100866

[76] Byrska-Bishop, M., Evani, U.S., Zhao, X., *et al.*: High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. Cell **185**(18), 3426–344019 (2022) https://doi.org/10.1016/j.cell.2022.08.004

[77] National Center for Biotechnology Information: ClinVar VCF Release (GRCh38). Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026 (2026). https://ftp.ncbi.nlm.nih.gov/pub/clinvar/vcf_GRCh38/

[78] Khan, Y., Saleem, M., Mehdi, M., Hogan, A., Mehmood, Q., Rebholz-Schuhmann, D., Sahay, R.: SAFE: SPARQL federation over RDF data cubes with access control. Journal of Biomedical Semantics **8**, 5 (2017) https://doi.org/10.1186/s13326-017-0112-6

[79] Staquet, D., Buelens, B., Bussche, J.: Incremental view maintenance for SPARQL queries: Adapting the counting algorithm. ACM Transactions on the Web (2026) https://doi.org/10.1145/3796549

[80] Corman, J., Florenzano, F., Reutter, J.L., Savković, O.: Validating SHACL constraints over a SPARQL endpoint. In: The Semantic Web – ISWC 2019. Lecture Notes in Computer Science, vol. 11778, pp. 145–163. Springer, ??? (2019). https://doi.org/10.1007/978-3-030-30793-6_9

[81] Figuera, M., Rohde, P.D., Vidal, M.-E.: Trav-SHACL: Efficiently validating networks of SHACL constraints. In: Proceedings of the Web Conference 2021 (WWW '21), pp. 3337–3348. ACM, ??? (2021). https://doi.org/10.1145/3442381.3449877

[82] Xiao, G., Ding, L., Cogrel, B., Calvanese, D.: Virtual knowledge graphs: An overview of systems and use cases. Data Intelligence **1**(3), 201–223 (2019) https://doi.org/10.1162/dint_a_00011

[83] NIST Genome in a Bottle (2025). https://www.nist.gov/programs-projects/genome-bottle Accessed 2025-11-24

[84] Crum, E., Buelens, B., Taelman, R., Ertaylan, G.: VCF-RDFizer evaluation: benchmark harness and run records. Zenodo. Dataset, published 2026-10-11. All versions: https://doi.org/10.5281/zenodo.23301185 (2026). https://doi.org/10.5281/zenodo.23301186 . https://doi.org/10.5281/zenodo.23301186

[85] Anthropic: Claude Opus 5.5 (Extra) [Large language model]. Accessed through the Claude Code desktop application. Used September 2026 (2026). https://www.anthropic.com/claude

[86] OpenAI: GPT-6 Astra (X-High) [Large language model]. Accessed through the Codex desktop application. Used September 2026 (2026). https://openai.com/